

Veille Prompt Engineering

Document de synthèse détaillé - P1IA - Groupe 1

Septembre 2026

Sommaire	Contenu
Ch. 1 à 3	LLM, introduction, structure d'un prompt
Ch. 4 à 6	Techniques de prompting, raisonnement, sorties structurées
Ch. 7 à 9	Applications métier, Machine Learning, RAG
Ch. 10 à 12	Évaluation, limites et risques, références

Accompagnement du dossier : **18 diapositives** (PowerPoint + HTML) dans 2-diapositives/ et **3 cheat sheets** dans 3-cheat-sheets/. Ce document est la version la plus complète : il détaille chaque notion du sujet avec du texte, des exemples et des tableaux.

1. Prérequis sur les LLM

Un **LLM** (Large Language Model) est un réseau de neurones géant entraîné à **prédire le prochain token** à partir du texte reçu. Sa compétence vient de l'entraînement sur des centaines de milliards de mots (livres, articles, code, forums...). Il ne possède pas une « base de connaissances » au sens classique : il calcule à chaque étape la distribution de probabilité du mot suivant.

1.1 Fonctionnement général

- **Contexte** : l'ensemble des tokens déjà présents (prompt + réponses précédentes). Le modèle ne fait que « continuer » ce contexte : sa qualité dépend donc directement de celle du contexte fourni.
- **Token** : unité de traitement. Un token peut être un mot court, un morceau de mot, un espace ou un signe de ponctuation. Ordre de grandeur : 100 tokens $\approx$ 75 mots, 1 page A4 $\approx$ 750 tokens.
- **Fenêtre de contexte** : limite technique du modèle en nombre de tokens (ex. 8 000, 32 000 ou 128 000 selon les modèles). Entrée + sortie doivent y tenir. Les modèles « oublient » ou rejettent ce qui dépasse ; certaines architectures traitent mieux le début que le milieu du contexte.
- **Generative** : un token à la fois, autrégressif, jusqu'à un token de fin, une *stop sequence*, ou l'atteinte de *max tokens*.

1.2 Paramètres qui pilotent la génération

Paramètre	Effet	Quand l'utiliser
Temperature	Basse = argmax quasi systématique : réponse stable, sèche, prédictible. Haute = échantillonnage plus hasardeux : varié, créatif.	0,0-0,5 : stable, exact, utile, court 0,7-1,0 : brainstorming, création, ton texte
Top-p (nucleus)	Ne garde que les tokens dont la probabilité cumulée atteint p. p=0,1 : 10% des probas cumulées.	Afin d'être le plus stable possible, p=0,1 est proche de la recommandation.
Top-k	Ne considère que les k tokens les plus probables (ex. 50).	Couper le « bruit » de fin de vocabulaire.
Max tokens	Longueur maximale de la sortie. Un résumé, un JSON ou un tableau sont dimensionnés en fonction du prompt.	Travailler sur des prompts si on est trop court par code que la réponse est tronquée.
Stop sequences	Chaînes qui interrompent la génération (ex. </response>, </>).	On définit de séparation un JSON ou un bloc de texte.

★ **À retenir** : Précision → temperature 0,2 ; créativité → 0,8. Prévoyez toujours une marge de max tokens et validez la fin de réponse (détection de troncature).

2. Introduction au Prompt Engineering

2.1 Qu'est-ce qu'un prompt ?

Un **prompt** est l'ensemble de ce que l'on envoie au modèle : consigne, question, texte à traiter, exemples, contraintes de format. Sans prompt, pas de réponse pertinente : c'est l'unique interface de contrôle avec un LLM.

2.2 Qu'est-ce que le Prompt Engineering ?

Le **Prompt Engineering** est la discipline qui regroupe la **conception** (rédaction), l'**évaluation** (mesure de qualité) et l'**optimisation itérative** des prompts. C'est une méthodologie, désormais adossée à des guides professionnels (OpenAI, Anthropic, Google, Microsoft) et à des publications scientifiques (CoT, ReAct, self-consistency...).

2.3 Pourquoi c'est important

- **Qualité** : deux formulations de la même demande peuvent donner un résultat excellent ou hors-sujet.
- **Rendement** : moins de reprises, moins de tokens gaspillés, latence et coût maîtrisés.
- **Fiabilité** : les formats contraints et les garde-fous réduisent hallucinations et erreurs d'exploitation.

- **Industrialisation** : un pipeline de prompts testé et versionné se déploie dans les applications métier en toute confiance.

Mauvais : Résume ce texte. Bon : Tu es un assistant qui rédige pour des cadres pressés. Résume le texte ci-dessous en 5 puces de 30 mots max, chaque puce commence par son mot-clé principal en gras.

3. Structure d'un prompt

Un prompt efficace s'assemble comme un formulaire : chaque brique remplit un rôle, l'ordre est libre mais la clarté prime. Les **délimiteurs** (guillemets, <<< >>>, balises XML, {json}) aident le modèle à distinguer consignes et données — et limitent les injections.

Brique	Rôle	Exemple
Rôle	Fixer le point de vue / l'expertise du modèle.	« Tu es un relecteur juridique. »
Contexte	Situer : public, finalité, contraintes de fond.	« Public : clients non techniques. »
Tâche	L'action, au verbe d'action unique.	« Relis et corrige les erreurs. »
Données d'entrée	Le matériau à traiter, délimité.	« Texte : <<<...>>> »
Contraintes	Longueur, ton, langue, interdictions.	« 15 lignes max, ton neutre, sans commentaire personnel. »
Format de sortie	Structure attendue du résultat.	« Tableau Markdown : erreur / correction / justification. »

3.1 Ordre et priorités

- Placer les **instructions** avant les **données** ; certaines architectures sont plus attentives au début du contexte.
- Répéter la contrainte clé (« uniquement en JSON ») en début ET en fin de prompt.
- Un seul objectif principal par prompt ; sinon découper (voir ch. 5).

Tu es un analyste financier. # ROLE Synthèse pédagogique pour un PDG. # CONTEXTE Résume ce rapport trimestriel. # TACHE Rapport : <<<texte>>> # DONNEES Max 8 lignes, zéro jargon. # CONTRAINTES 3 puces + 1 conclusion en italique. # FORMAT

★ **À retenir** : Vérifiez chaque brique : si l'une manque, la réponse est souvent imprécise. Le format de sortie est la brique la plus souvent oubliée et la plus rentable.

4. Techniques de prompting

4.1 Zero-shot prompting

Aucun exemple. On décrit la tâche et on fournit l'entrée. Idéal pour les tâches « naturelles » que le modèle connaît déjà très bien. Minimum de tokens.

Classe le sentiment de ce commentaire (positif / négatif / neutre) : « Le produit est arrivé cassé. »

4.2 One-shot fondamental

Un exemple unique montre le format, la structure ou le ton attendu. Utilisé pour forcer un format, un style ou une langue cible à moindre coût.

Convertis un titre de film en emojis. Exemple : « Jaws » -> [requin][vague] « Titanic » -> ?

4.3 Few-shot prompting

Deux à cinq exemples (parfois jusqu'à une dizaine) montrent la logique de transformation. C'est la meilleure façon d'établir un format nouveau, une classification fine ou un style particulier. Les exemples doivent être **représentatifs et variés** (y compris les cas limites).

Classe chaque e-mail : Travail / Personnel / Spam. « -50% ce week-end uniquement ! » -> Spam
« Peux-tu m'envoyer le budget Q2 ? » -> Travail « Recette de tarte de ma grand-mère » -> ?

4.4 Chain-of-Thought (CoT)

Au lieu de demander une réponse directe, on demande un **raisonnement visible étape par étape**. Publié par Wei et al. (2022), le CoT améliore nettement l'arithmétique, la logique, la planification. Deux versions : le **CoT zéro-shot** (« raisonne étape par étape ») et le **CoT few-shot** (on fournit des raisonnements détaillés en exemple).

Un train quitte Paris 9h à 120 km/h, un second quitte Lyon 9h à 90 km/h. Distance Paris-Lyon : 470 km. À quelle heure se croisent-ils ? Raisonne étape par étape.

4.5 Techniques avancées de raisonnement

Technique	Principe	Quand l'utiliser
Self-consistency (Wang et al. 2022)	Génère plusieurs raisonnements CoT (temperature élevée)	Problèmes à réponse unique mais plusieurs chemins possibles (CoT)
Tree-of-Thought (Yao et al. 2023)	Explique plusieurs branches de raisonnement comme un arbre	Problèmes de planification et de jeu à terme
Step-back prompting	D'abord demander le principe général, puis l'appliquer	Questions de physique/maths/politique : remonter aux fondamentaux
ReAct (Yao et al. 2022)	Alterner Raisonnement et Actions (appel d'outils : calculatrice, recherche, API)	Tâches nécessitant des informations externes ou des calculs (agents)
APE - Automatic Prompt Engineering	Demander au LLM de générer plusieurs prompts candidats	Optimisation des prompts en production

4.6 Structured prompting

La sortie est contrainte à un format strict (JSON, XML, tableau, sections numérotées). C'est la passerelle entre le LLM et les programmes : elle est détaillée au chapitre 6.

Réponds exclusivement en JSON, sans texte autour : { "resume": "...", "mots_cles": [...], "score_confiance": 0.0-1.0 }

Technique	Coût en tokens	Précision	Quand privilégier
Zero-shot	Minimal	Bonne (tâches simples)	Réponses directes, budget serré
One-shot	Faible	Bonne (format)	Forcer un format/style
Few-shot	Moyen	Élevée	Classification, formats exotiques
CoT	Moyen	Élevée (raisonnement)	Logique, calcul, méthode
Self-consistency	Fort (N réponses)	Très élevée	Décision à enjeu, réponse unique
Structured	Moyen	Élevée (format)	Exploitation par un programme

★ **À retenir** : Commencez en zéro-shot. Ajoutez un exemple si le format dérive, passée en few-shot si la précision ne suit pas, et combinez avec CoT pour les tâches de raisonnement.

5. Prompt Engineering et raisonnement

5.1 Prompt decomposition

Plutôt qu'un prompt monolithique, on **décompose** en étapes : chaque étape est simple à vérifier, et l'assemblage est contrôlé. En pratique, on peut tenir la décomposition dans un seul prompt (instructions numérotées) ou laisser un orchestrateur appeler le LLM plusieurs fois (pipeline).

Étape 1 : extrais la liste des produits et leur prix. Étape 2 : pour chaque produit, calcule la marge. Étape 3 : classe par marge décroissante puis rends un tableau. Réponds étape par étape.

5.2 Self-check et vérification

Les LLM améliorent souvent leur propre production quand on leur demande de se relire avec un angle critique : on parle de **self-refinement**. Variante plus robuste : produire deux réponses, demander une critique croisée, puis fusionner. C'est un cas particulier du pattern plus général « double-pass ».

1) Propose une réponse. 2) Relis-la en tant que correcteur strict : liste 3 erreurs ou oublis. 3) Réécris la réponse en corrigeant ces points.

★ **À retenir** : Pour la production : décomposez + faites relire. Pour la décision critique : croisez deux réponses indépendantes. Le surcoût en tokens paye largement en fiabilité.

6. Sorties structurées

6.1 Pourquoi structurer ?

- **Automatisation** : le JSON s'injecte directement dans une base ou une API, sans passage fragile.
- **Comparabilité** : deux réponses sur le même schéma se testent et se notent.
- **Complétude** : la structure matérialise les champs obligatoires → moins d'oublis.

6.2 Formats

- **JSON / JSON schema** : passage de standard applications ; le schéma décrit types et champs (validé par code).
- **XML** : documentaire, catalogues, interopérabilité.
- **Markdown / CSV** : lectures humaines et exports tableurs.

6.3 Validation

- Vérifier **par le code** (json.loads, diff de schéma), **jamais par le modèle seul**.
- Demander au modèle de **relire face au schéma** mentionné dans le prompt.
- En cas d'échec : relancer avec un prompt renforcé (squelette fourni, temperature basse).

```
{ "type": "object", "required": ["resume", "mots_cles"], "properties": { "resume": { "type": "string"}, "mots_cles": { "type": "array", "items": { "type": "string"}}, "score_confiance": { "type": "number", "minimum": 0, "maximum": 1} } }
```

7. Prompts pour les applications métier

Application	Prompt type	Piège classique
Résumé de documents	« Résume en 3 puces en conservant les chiffres clés. »	Résumé trop long ou invente des chiffres → fixer longueur + ex
Traduction	« Traduis en anglais professionnel ; conserve les nombres et la ponctuation. »	Perte de registres et précision → registre + glossaire
Classification	« Classe ce ticket : bug / question / feature. »	Catégories ouvertes → fournir une liste close et un cas limite e
Extraction d'informations	« Extrais nom, date, montant ; réponds en JSON. »	Format libre → schéma JSON + valeur par défaut « null » si ab
Génération de texte	« Mail de relance de paiement au fournisseur X : 150 mots, ton professionnel, date limite de paiement : 15/05/2024. »	Réponse générique, date limite de paiement + contraintes + exen
Génération de code	« Fonction Python qui calcule la moyenne d'une liste ; donne la fonction et la liste de test. »	Cas limite non testés → où spécifier les cas limites et den

★ **À retenir** : Règle d'or métier : le prompt est un **cahier des charges**. Public + longueur + ton + format + exemples transforme une réponse moyenne en livrable utilisable.

8. Prompts pour le Machine Learning

8.1 Préparation des données

Nettoie ce jeu de données : supprime les doublons, corrige les valeurs abérantes, normalise les dates ; rends un tableau de chaque correction avec sa justification.

8.2 Génération de visualisations

Génère le code Python (matplotlib) d'un histogramme du CA mensuel à partir de ces données. Étiquettes en français, taille 800x500, pas de texte hors code.

8.3 Sélection de modèles

Objectif : classer des e-mails spam/non-spam, 10 000 exemples. Compare 3 familles de modèles et recommande en arbitrart précision / temps d'entraînement / coût de mise en prod.

8.4 Interprétation des métriques

Matrice de confusion : précision 0,72 / rappel 0,88. Explique le sens en dépistage médical (FP, FN) puis propose 2 pistes d'amélioration justifiées.

★ **À retenir** : Constante utile en ML : demander systématiquement le **code ET l'explication**. Le code est exécutable à vérifier, l'explication est le raisonnement à valider.

9. Prompt Engineering et RAG

9.1 Pourquoi le prompting seul ne suffit pas

- Le LLM est figé à sa date d'entraînement : pas vos documents internes, pas l'actualité.
- Sans source, impossible de **vérifier** la réponse : les hallucinations passent inaperçues.
- La fenêtre de contexte ne peut pas contenir une base entière de documents.

9.2 Le RAG en 3 étapes

- **1. Indexation** : les documents sont découpés en morceaux (chunks) et convertis en **embeddings** (vecteurs de sens).
- **2. Recherche** : à la question, on cherche les chunks les plus proches (similarité cosinus) dans la base vectorielle.
- **3. Génération** : les chunks retenus sont insérés dans le prompt, et le modèle répond *uniquement* sur eux, en citant les sources.

9.3 Construction d'un prompt RAG

Réponds UNIQUEMENT à partir des extraits ci-dessous. Si l'information n'y figure pas, réponds exactement : « Information introuvable dans les documents. » Extraits : <<<doc1 : ...>>> <<<doc2 : ...>>> Question : ... Réponds en français, 200 mots max, en citant la source de chaque affirmation.

Piège RAG	Conséquence	Correctif
Chunk mal découpé	Informations coupées en deux	Chunking par paragraphes/sections, fenêtre avec chevauchement
Top-k trop petit	Information pertinente absente	Augmenter top-k, hybride BM25 + vecteurs
Extraits confusants	Réponse désorientée	Encadrer chaque extrait, numéroter, dédupliquer
Absence de garde-fou	Réponse hors des documents	Instruction explicite « réponds uniquement sur les extraits »

★ **À retenir** : Le RAG rend chaque affirmation **traçable**. Vérifiez toujours (méthode d'évaluation) que la réponse cite bien un extrait existant.

10. Évaluation et optimisation des prompts

10.1 Pourquoi évaluer ?

Un prompt long, cher et « oublié » qui marche sur un exemple peut échouer sur 30 % des entrées réelles. L'évaluation mesure la qualité de façon **reproductible** avant mise en production.

10.2 Critères et méthodes de mesure

Méthode	Description	Usage typique
Grille de notation manuelle	De 1 à 5 sur pertinence, format, exhaustivité ; score O/N hallucinations	Revisions, avis qualitatif, arbitrage
Jeu de test fixe (gold set)	Questions + réponses attendues ; % de réussite.	Régression lors des modifications de prompt
Exact match / F1	Comparaison stricte ou soft avec la réponse de référence	Extraction, classification
BLEU / ROUGE	Mesures lexicales entre sortie et référence.	Résumé, traduction (usage décroissant avec l'IA)
LLM-as-judge	Un LLM note ou compare la réponse selon un barème.	Gros volumes, subjectivité, comparer 2 prompts

10.3 Comparaison de résultats

- **A/B testing** : deux prompts sur les mêmes entrées, sorties présentées côte à côte ou notées à l'aveugle.
- Faire varier **un seul paramètre à la fois** (autrement impossible d'attribuer un effet).

10.4 Optimisation itérative

Cycle : écrire le prompt -> tester le même échantillon -> catégoriser les erreurs (contenu, format, ton) -> corriger -> re-tester. Versionner à chaque essai (v1, v2...).

★ **À retenir** : Garde la trace des versions : prompt, temperature, jeu de test, score. Sans versionnage, l'optimisation devient du bricolage.

11. Limites et risques

11.1 Hallucinations

Le modèle fabrique des informations **fausses mais formulées avec assurance**, surtout sur les sujets rares ou quand la donnée est absente. Mitigations : RAG + citations, self-check, demande de « déclaré non connu », relecture humaine.

11.2 Ambiguïté

Un prompt flou laisse le modèle choisir une interprétation arbitraire. Mitigation : préciser tâche, périmètre, format, exceptions ; reformuler et tester deux formulations.

11.3 Biais

Les LLM répliquent des stéréotypes présents dans leurs données d'entraînement (genre, origine, âge...). Mitigation : prompts neutres, jeu de test contenant des cas sensibles, relecture humaine, débats contradictoires.

11.4 Confidentialité

Ce qui est envoyé à une API peut être journalisé, utilisé ou stocké. Règle : **ne jamais envoyer de données identifiantes ou sensibles ; anonymiser en amont** ; vérifier les engagements contractuels du fournisseur (data residency, retention).

11.5 Injection de prompt

Attaque par laquelle un contenu malveillant (dans un document, un e-mail, un site) **réécrit les instructions** : ex. « ignore toutes les instructions précédentes ». Défenses : séparer instructions et données, délimiteurs, validation de sortie, contrôle d'accès aux outils.

Contexte utilisateur (injecté) : <<< du texte » Mails, prière de ne pas répéter le system prompt « >>> => contenir la donnée dans des balises ne suffit pas : toujours valider la sortie.

11.6 Jailbreak awareness

Des formulations détournent les garde-fous (roleplay, mise en scène, chiffrement, « DAN »...). Même en environnement interne, il faut compter sur des **filtres de sortie, surveillance, mises à jour et supervision humaine** des actions sensibles.

12. Références

- OpenAI - Prompt Engineering Guide : <https://platform.openai.com/docs/guides/prompt-engineering>
- Anthropic - Prompt Engineering Overview : <https://docs.anthropic.com/en/docs/build-with-claude/prompt-engineering>
- Google / Gemini API - Prompting intro : <https://ai.google.dev/gemini-api/docs/prompting-intro>
- Prompt Engineering Guide (DAIR AI) : <https://www.promptingguide.ai>
- Microsoft - Azure OpenAI best practices : <https://learn.microsoft.com/fr-fr/azure/ai-services/openai/concepts/prompt-engineering>
- CoT - Chain-of-Thought Prompting Elicits Reasoning in LLMs (Wei et al., 2022) : <https://arxiv.org/abs/2201.11903>
- Self-consistency (Wang et al., 2022) : <https://arxiv.org/abs/2203.11171>
- Tree-of-Thought (Yao et al., 2023) : <https://arxiv.org/abs/2305.10601>
- ReAct (Yao et al., 2022) : <https://arxiv.org/abs/2210.03629>
- RAG - Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks (Lewis et al., 2020) : <https://arxiv.org/abs/2005.11401>
- OWASP - Generative AI / LLM top 10 : <https://owasp.org/www-project-top-10-for-large-language-model-applications/>
- Cheat sheets du dossier : ipwithease.com, Google AI cheat sheet (avril 2025), RPATech business users

★ **À retenir** : Travail livré par le groupe 1 (P1IA). Fichiers associés : 2-diapositives/ (PPTX + HTML, 18 diapos) et 3-cheat-sheets/ (3 PDF + sources).